

Contents

Подготовка к экзамену по проекту	1
«Монте-Карло оценка девелоперского проекта: NPV и корреляция рисков»	1
1. Краткая суть проекта	1
2. Для кого проект	2
3. Какая проблема решается	2
4. Теоретическая база	3
5. Архитектура проекта	4
6. Логика работы dashboard пошагово	5
7. Подробный разбор model.py	5
8. Подробный разбор app.py	7
9. Подробный разбор utils.py	8
10. Разбор графиков	8
11. Интерпретация результатов	10
12. Как защищать проект преподавателю (демонстрация)	10
13. Сильные тезисы для защиты (10-15)	11
14. Возможные вопросы преподавателя и ответы (30)	11
15. Ограничения модели	13
16. Как улучшить проект	13
17. Шпаргалка на 1 минуту	13

Подготовка к экзамену по проекту

«Монте-Карло оценка девелоперского проекта: NPV и корреляция рисков»

Автор: Левшин Даниил Дмитриевич **Дисциплина:** «Инвестиционная оценка»
Формат: интерактивный Streamlit-dashboard (прикладное продолжение курсовой работы) **Москва, 2026**

Этот файл самодостаточен: по нему можно понять всю логику проекта — финансовую идею, математику и устройство кода — даже не открывая исходники.

1. Краткая суть проекта

Проект — это **интерактивный веб-дашборд**, который оценивает девелоперский (строительный) проект не одним числом, а распределением возможных исходов.

Простыми словами:

- это **интерактивный dashboard**, который открывается в браузере;
- он оценивает **девелоперский проект** — строительство и продажу недвижимости;
- использует классическую метрику **NPV** (чистая приведённая стоимость) и метод **Монте-Карло**;
- показывает **не одно значение NPV, а распределение** из тысяч возможных исходов;

- сравнивает две модели — с **независимыми** и с **коррелированными** рисками;
- помогает понять, **насколько опасно игнорировать корреляцию рисков** при оценке проекта.

Главная идея в одной фразе: проект переводит вопрос «**чему равна NPV проекта?**» в вопрос «**с какой вероятностью проект создаст или разрушит стоимость?**».

2. Для кого проект

Группа	Зачем им инструмент
Девелоперы	Оценить не только ожидаемую доходность, но и вероятность убытка до старта стройки; понять, какой фактор сильнее всего угрожает проекту.
Инвесторы	Принять решение о входе в проект, опираясь на распределение исходов и риск-метрики (VaR, ES), а не на единственную точечную оценку.
Банки проектного финансирования	Оценить кредитный риск проекта: вероятность того, что проект не обслужит долг, и глубину потерь в стрессовых сценариях.
Оценщики	Получить вероятностную оценку стоимости с доверительными интервалами вместо одного значения.
Консультанты	Наглядно показать клиенту чувствительность проекта к ключевым факторам и эффект корреляции рисков.
Студенты и преподаватели	Демонстрация практического применения метода Монте-Карло, гауссовой копулы и риск-метрик в инвестиционной оценке.

3. Какая проблема решается

Почему обычный (точечный) NPV недостаточен. Базовый NPV считается при «наиболее вероятных» значениях параметров и даёт одно число. Оно ничего не говорит о том, насколько проект рискован: два проекта с одинаковым $NPV = +200$ млн могут иметь совершенно разный разброс исходов — один почти гарантированно прибыльный, другой балансирует на грани убытка.

Почему сценарный анализ ограничен. Классический подход «оптимистичный / базовый / пессимистичный» рассматривает 3–5 сценариев. Но он не даёт вероятностей, субъективен в выборе сценариев и не показывает всё пространство исходов между крайними точками.

Почему Monte Carlo лучше для оценки неопределённости. Метод Монте-Карло генерирует тысячи случайных сценариев согласно заданным распределениям факторов и строит **полное распределение NPV**. Это позволяет рассчитать вероятность убытка, перцентили, VaR и Expected Shortfall — то есть количественно измерить риск.

Почему строительные, рыночные и регуляторные риски нельзя считать независимыми. В девелопменте плохие события приходят «пачкой»: рост стоимости стройматериалов часто совпадает с задержкой стройки; падение рынка снижает цену продажи и одновременно замедляет темп продаж; ужесточение регулирования увеличивает и расходы, и сроки. Считать эти факторы независимыми — значит занижать вероятность одновременного наступления нескольких бед.

Как корреляция меняет левый хвост распределения NPV. Положительная корреляция между неблагоприятными факторами делает совместное наступление плохих событий более вероятным. Это «утяжеляет левый хвост» распределения NPV: растёт вероятность убытка ($P(NPV < 0)$), ухудшаются VaR и Expected Shortfall, хотя средняя NPV может почти не измениться. Именно этот эффект — главный результат, который демонстрирует проект.

4. Теоретическая база

NPV (чистая приведённая стоимость) — сумма дисконтированных денежных потоков проекта за вычетом начальных инвестиций. Положительная NPV означает, что проект создаёт стоимость.

$$NPV = \sum CF_t / (1 + r)^t - I_0 \quad (t = 1 \dots T)$$

где CF_t — чистый денежный поток года t , r — ставка дисконтирования, I_0 — начальные инвестиции, T — горизонт проекта.

DCF (Discounted Cash Flow) — метод оценки, при котором будущие потоки приводятся к настоящему моменту через дисконтирование. NPV — итоговый показатель DCF-модели.

Денежный поток девелоперского проекта. Притоки — выручка от продажи недвижимости (GDV, Gross Development Value). Оттоки — строительные затраты, операционные/административные расходы, регуляторные расходы, расходы на содержание при задержке. Потоки распределены по годам согласно профилям (строительство — раньше, выручка — позже).

Monte Carlo simulation — численный метод: многократная (10 000+) генерация случайных значений факторов и расчёт NPV для каждого набора. Результат — выборка из распределения NPV.

Распределения вероятностей задают, как может меняться каждый фактор риска.

Треугольное распределение $Tri(min, mode, max)$ — задаётся тремя числами: минимум, наиболее вероятное значение, максимум. Удобно для экспертных оценок, когда известны границы и «центр». Используется для затрат, цены, регуляторных расходов и темпа продаж.

Дискретное распределение — конечный набор исходов с вероятностями. Используется для задержки строительства: 0 мес. (50%), 6 мес. (35%), 12 мес. (15%).

Корреляционная матрица — симметричная матрица 5×5 с единицами на диагонали, элемент ρ_{ij} показывает силу и знак линейной связи между факторами i и j .

Гауссова копула — способ связать несколько распределений с заданной корреляционной структурой. Сначала генерируются коррелированные нормальные величины, затем они преобразуются в нужные предельные распределения через функцию распределения и обратные функции.

Разложение Холецкого — представление корреляционной матрицы Σ в виде $\Sigma = L \cdot L^T$, где L — нижнетреугольная матрица. Умножая независимые нормальные величины на L^T , получаем коррелированные. Требуется, чтобы матрица была положительно определённой.

VaR (Value at Risk, 5%) — уровень NPV, ниже которого результат окажется лишь с вероятностью 5%. Технически — 5-й перцентиль распределения NPV.

VaR 5% = 5-й перцентиль распределения NPV

Expected Shortfall (ES, ожидаемые потери) — средняя NPV в худших 5% сценариев. Показывает не границу, а среднюю глубину потерь в зоне катастрофы.

Expected Shortfall = среднее значение NPV в худших 5% сценариев

Sensitivity analysis (анализ чувствительности) — оценка того, насколько сильно итоговая NPV зависит от каждого фактора. В проекте измеряется коэффициентом корреляции Пирсона между значением фактора и NPV по всем итерациям.

Tornado chart (торнадо-диаграмма) — горизонтальная столбчатая диаграмма, ранжирующая факторы по силе влияния на NPV (самый влиятельный — сверху). Форма напоминает воронку торнадо.

5. Архитектура проекта

Проект построен по принципу **разделения ответственности**: интерфейс, математика и вспомогательные функции живут в разных модулях. Это делает код читаемым, тестируемым и пригодным для повторного использования.

Файл	Роль
app.py	Интерфейс Streamlit : ввод параметров, кнопка запуска, отображение графиков и таблиц. Только UI и визуализация — без расчётной математики.
model.py	Расчётное ядро : базовый NPV, симуляция Монте-Карло, генерация рисков (копула + Холецкий), риск-метрики, чувствительность. Не зависит от Streamlit.

Файл	Роль
<code>utils.py</code>	Вспомогательные функции: форматирование чисел, проверка корреляционной матрицы, построение таблиц, генерация текстового инвестиционного вывода.
<code>requirements.txt</code>	Список зависимостей (streamlit, numpy, pandas, scipy, plotly).
<code>README.md</code>	Описание проекта: для кого, как запустить, как развернуть, методология, ограничения.
<code>.streamlit/config.toml</code>	Настройки внешнего вида (светлая тема, корпоративные цвета, шрифт).

Зачем разделение. `model.py` можно импортировать и тестировать отдельно от интерфейса (что и сделано — у него есть самотест в блоке `if __name__ == "__main__"`). Если завтра нужно сделать не веб-интерфейс, а отчёт или API — математику менять не придётся. Это профессиональный, а не «учебно-игрушечный» подход.

6. Логика работы dashboard пошагово

Шаг 1. Базовые параметры проекта (боковая панель): начальные инвестиции I_0 , базовая выручка (GDV), строительные затраты, операционные расходы, ставка дисконтирования, горизонт проекта, число итераций, `seed`.

Шаг 2. Параметры рисков (раздел 3 интерфейса): рост строительных затрат, изменение цены продажи, задержка строительства, регуляторные расходы, темп продаж — для каждого задаются параметры распределения.

Шаг 3. Корреляционная матрица (раздел 4): пользователь может отредактировать коэффициенты в таблице.

Шаг 4. Проверка корректности матрицы: программа проверяет положительную определённость; при некорректном вводе использует ближайшую корректную матрицу и выводит предупреждение.

Шаг 5. Запуск двух симуляций: с независимыми рисками и с коррелированными рисками.

Шаг 6. На каждой итерации: генерируются случайные значения факторов → рассчитываются денежные потоки по годам → дисконтируются → считается NPV.

Шаг 7. После всех итераций: строится распределение NPV, рассчитываются риск-метрики (среднее, медиана, VaR, ES, $P(NPV < 0)$, перцентили), строятся графики, формируется автоматический инвестиционный вывод.

7. Подробный разбор `model.py`

Это расчётное ядро. Ключевые структуры данных и функции:

Структуры данных (dataclass)

ProjectParams — базовые параметры проекта: `initial_investment` (500), `base_revenue` (1850), `base_construction` (550), `opex` (50), `discount_rate` (0.16), `horizon` (4), профили потоков по годам (`construction_profile`, `revenue_profile`, `opex_profile`), `delay_carry_per_year` (40 — расходы на содержание за год задержки).

RiskParams — параметры предельных распределений факторов: `cost_growth` (-0.05, 0.10, 0.30), `price_change` (-0.20, 0.05, 0.20), `delay_values/delay_probs` (0/6/12 мес. с вероятностями 0.50/0.35/0.15), `reg_cost` (0, 20, 80), `absorption` (-1, 0, 1).

Зачем нужны dataclass: они группируют десятки параметров в два понятных объекта, дают значения по умолчанию, делают сигнатуры функций короткими (передаём один `params` вместо 10 аргументов) и документируют модель. Это чище и безопаснее, чем десятки разрозненных переменных.

Ключевые функции

calculate_base_npv(params) — базовая (детерминированная) NPV. *Вход:* `ProjectParams`. *Выход:* одно число (NPV в млн руб.). *Логика:* вызывает `calculate_npv` при нейтральных (нулевых) значениях всех факторов. *Связь с оценкой:* это «классический» точечный NPV, отправная точка, относительно которой видно смещение распределения.

calculate_npv(cost_growth, price_change, delay_months, reg_cost, absorption, params) — векторизованный расчёт NPV. *Вход:* массивы реализаций пяти факторов (длиной `N`) и параметры проекта. *Выход:* массив NPV длиной `N`. *Логика:* цена масштабирует выручку $REV = base_revenue \cdot (1 + price_change)$; рост затрат масштабирует стройку; регуляторные расходы добавляются в 1-й год; задержка сдвигает дисконтирование выручки на `delay_months/12` лет и добавляет расходы на содержание; темп продаж (`absorption`) перераспределяет выручку между ранним и поздним годами. Затем потоки дисконтируются и суммируются. *Связь:* это сердце DCF — формула NPV, применённая сразу ко всем сценариям (векторно, через NumPy, поэтому быстро).

generate_correlated_risks(corr_matrix, risk, n, seed, use_correlation) — генерация факторов риска (гауссова копула). *Вход:* корреляционная матрица, параметры рисков, число сценариев, `seed`, флаг учёта корреляции. *Выход:* словарь массивов значений факторов (`cost`, `price`, `delay`, `reg`, `absorption`) и матрица $N \times 5$. *Логика (копула):* 1. генерируем независимые стандартные нормальные $Z \sim N(0, I)$; 2. если `use_correlation=True` — применяем Холецкого: $L = \text{cholesky}(corr)$, $Z^* = Z \cdot L^T$ (теперь нормальные коррелированы); 3. переводим в равномерные $U = \Phi(Z^*)$ (функция нормального распределения); 4. через обратные функции предельных распределений получаем значения факторов. *Связь:* это механизм, который «склеивает» риски с заданной корреляцией — ядро всей идеи проекта.

Генерация независимых рисков — та же функция с `use_correlation=False`: шаг с Холецким пропускается, факторы остаются независимыми. Это базовая (наивная) модель для сравнения.

Применение корреляционной матрицы и Cholesky decomposition — именно строки `L = np.linalg.cholesky(corr_matrix)` и `z = z @ L.T`. Холецкий «впечатывает» корреляционную структуру в нормальные величины: ковариация Z^* равна $L^T \cdot L = \Sigma$.

Обратные функции распределений (PPF): - `_triangular_ppf(u, a, c, b)` — обратная функция треугольного распределения (замкнутая формула); - `_discrete_ppf(u, values, probs)` — обратная функция дискретного распределения (для задержки).

`run_monte_carlo(params, risk, corr_matrix, n, seed, use_correlation)` — полная симуляция одной модели: генерирует риски и считает массив NPV. Возвращает NPV, факторы и базовую NPV.

`run_both_models(params, risk, corr_matrix, n, seed)` — запускает обе модели: коррелированную (`seed`) и независимую (`seed+1`, чтобы выборки не совпадали тождественно) и возвращает для каждой NPV, факторы, метрики и чувствительность.

`calculate_risk_metrics(npv, base_npv, alpha=0.05)` — риск-метрики. *Выход:* словарь: `mean, median, std, min, max, p5, p95, prob_neg (%)`, `var, es, base`. *Расчёт VaR:* `p5 = np.percentile(npv, 5)` — 5-й перцентиль. *Расчёт Expected Shortfall:* сортируем NPV, берём худшие `k = 5% · N` значений, считаем их среднее: `npv_sorted[:k].mean()`. *P(NPV<0):* доля сценариев с отрицательной NPV.

`calculate_sensitivity(factors, npv)` — чувствительность. *Логика:* для каждого фактора считается коэффициент корреляции Пирсона `np.corrcoef(col, npv)` между значением фактора и итоговой NPV; результат сортируется по модулю корреляции. *Выход:* список `{factor, corr, direction}`. *Связь:* отвечает на вопрос «какой фактор сильнее всего двигает NPV».

Численная основа без SciPy. Модуль работает и без SciPy: функция нормального распределения реализована через `math.erf`, а обратная — через рациональную аппроксимацию Акклма. Это делает ядро самодостаточным и тестируемым где угодно.

8. Подробный разбор `app.py`

Это интерфейс. Где что происходит:

- **Настройка страницы** — `st.set_page_config(...)` в начале: заголовок, иконка, широкий layout. Далее — CSS-стили для карточек и оформления.
- **Параметры sidebar** — блок `sb = st.sidebar`: `st.number_input/st.slider/st.select_slider` для инвестиций, выручки, затрат, OPEX, ставки, горизонта, числа итераций и `seed`.
- **Поля ввода рисков** — раздел 3: `st.number_input` для границ треугольных распределений и `st.slider` для вероятностей задержки. Из них собирается объект `RiskParams`.
- **Редактируемая корреляционная матрица** — раздел 4: `st.data_editor` с матрицей; результат проверяется `utils.validate_correlation_matrix(...)`, при некорректности выводится `st.warning`.
- **Запуск симуляции** — раздел 5: кнопка `run` в сайдбаре; при нажатии вызывается `model.run_both_models(...)`, результат кладётся в `st.session_state["results"]`.
- **Зачем `session_state`.** Streamlit перезапускает весь скрипт при каждом действии пользователя (например, наведении на график). `session_state` хранит результаты симуляции между перезапусками, чтобы не пересчитывать 10 000

итераций при каждом клике и чтобы графики не «исчезали». Если результата ещё нет — `st.stop()` прекращает отрисовку до запуска.

- **KPI cards** — раздел 6: HTML-карточки `metric_card(...)` со средней NPV, медианой, $P(NPV < 0)$, VaR.
- **Гистограмма** — `fig_histogram(...)` (Plotly): распределение NPV с линиями $NPV=0$, среднего и VaR.
- **CDF** — `fig_cdf(...)`: кумулятивные кривые для двух моделей.
- **Сравнение распределений** — `fig_compare_hist(...)`: наложенные гистограммы независимой и коррелированной моделей; рядом — сравнительная таблица `utils.metrics_table(...)`.
- **Heatmap** — `fig_heatmap(...)`: тепловая карта корреляционной матрицы.
- **Tornado chart** — `fig_tornado(...)`: чувствительность NPV к факторам.
- **Инвестиционный вывод** — раздел 9: `utils.generate_investment_verdict(...)` формирует цветной блок с вердиктом и эффектом корреляции.
- **Методология** — раздел 10: формулы (`st.latex`) и описание модели.

Логика для устного ответа: «Вот **здесь** (сайдбар) пользователь меняет параметры, **здесь** (разделы 3–4) задаёт риски и корреляцию, по кнопке вызывается **расчётная модель** `model.run_both_models`, а **здесь** (разделы 6–9) результаты рисуются графиками Plotly и формируется вывод».

9. Подробный разбор `utils.py`

- **Форматирование:** `fmt_money` (число → «1 234,5 млн руб.»), `fmt_num`, `fmt_pct` — единообразный русскоязычный формат с пробелом-разделителем разрядов и запятой.
- **Таблицы:** `metrics_table` (сравнение двух моделей), `sensitivity_table` (чувствительность), `correlation_dataframe` (матрица с подписями), `scenario_table`.
- **Проверка корреляционной матрицы:** `validate_correlation_matrix(matrix, fallback)` — симметризует матрицу, фиксирует диагональ = 1, проверяет положительную определённость через разложение Холецкого; если матрица некорректна — строит ближайшую корректную через `_nearest_psd` (клиппинг отрицательных собственных значений + нормировка диагонали) и возвращает сообщение для интерфейса. `is_positive_definite` — быстрая проверка.
- **Текстовый вывод:** `generate_investment_verdict(metrics_corr, metrics_ind)` — классифицирует проект (привлекательный / умеренный / рискованный) по средней NPV и вероятности убытка, формирует заголовок, развёрнутый текст и отдельный блок про эффект учёта корреляции.

Почему вынесено отдельно. Эти функции — не математика модели и не интерфейс, а «сервис» между ними. Вынос в отдельный модуль не загромождает `app.py`, позволяет тестировать форматирование и проверку матрицы независимо и переиспользовать их (например, в отчёте или другом интерфейсе).

10. Разбор графиков

График	Ось X	Ось Y	Управленческий смысл	Что говорит преподавателю
Гистограмма NPV	NPV, млн руб.	Частота (число итераций)	Форма распределения исходов, где сосредоточена масса, насколько широк разброс	«Вместо одного числа мы видим всё облако исходов; красная линия $NPV=0$ отделяет прибыль от убытка, серая — VaR ».
CDF (кумулятивная функция)	NPV, млн руб.	$P(NPV \leq x)$	Вероятность того, что NPV не превысит заданный уровень	«По кривой я сразу читаю вероятность убытка — это высота кривой в точке $NPV=0$; коррелированная кривая левее — риск выше».
Сравнение распределений	NPV, млн руб.	Частота	Наглядная «цена игнорирования корреляции»	«Синее (корреляция) распределение имеет более тяжёлый левый хвост, чем серое (независимость)».
Heatmap корреляций	Факторы риска	Факторы риска	Структура взаимосвязей рисков	«Тёплые клетки — положительная связь, холодные — отрицательная; именно эта структура впечатывается в симуляцию через Холецкого».

График	Ось X	Ось Y	Управленческий смысл	Что говорить преподавателю
Tornado chart	Корреляция фактора с NPV	Факторы (по убыванию влияния)	Ранжирование факторов по важности	«Самый длинный столбец сверху — фактор, на котором нужно сосредоточить управление рисками».

11. Интерпретация результатов

- **Положительная средняя NPV** означает, что в среднем проект создаёт стоимость — но это лишь центр распределения.
- **Почему положительная средняя NPV не гарантирует безопасность:** среднее ничего не говорит о разбросе. При средней +110 млн вероятность убытка может оставаться 20–25% — четверть сценариев убыточны.
- **Вероятность $NPV < 0$** — прямая мера риска: доля сценариев, в которых проект разрушает стоимость.
- **Почему важен VaR:** он показывает порог потерь, который не будет превышен с вероятностью 95% — стандартный язык риск-менеджмента и банков.
- **Почему ES важнее VaR в стрессовой зоне:** VaR — это граница («хуже этого с вероятностью 5%»), но он не говорит, насколько глубоко может провалиться проект за этой границей. Expected Shortfall усредняет именно катастрофические 5% сценариев и показывает их среднюю глубину — это более консервативная и информативная метрика.
- **Как корреляция влияет на левый хвост:** положительные связи между неблагоприятными факторами повышают вероятность их одновременного наступления → левый хвост тяжелеет → растёт $P(NPV < 0)$, ухудшаются VaR и ES.
- **Почему независимая модель занижает риск:** игнорируя совместное наступление бед, она «размазывает» риск и показывает более оптимистичную (заниженную) вероятность убытка.

12. Как защищать проект преподавателю (демонстрация)

1. Открыть dashboard в браузере.
2. Показать **базовые параметры** в сайдбаре и базовую NPV.
3. Объяснить **пять факторов риска** и их распределения.
4. Показать **корреляционную матрицу** и heatmap, объяснить смысл связей.
5. Нажать «Запустить симуляцию».
6. Показать **распределение NPV** (гистограмму) и объяснить линии.
7. Показать **вероятность убытка** и VaR/ES в KPI-картах.

8. Показать **сравнение независимой и коррелированной** моделей (наложенные гистограммы + CDF + таблица) — ключевой момент защиты.
 9. Показать **tornado chart** — какой фактор важнее всего.
 10. Зачитать **автоматический инвестиционный вывод**.
-

13. Сильные тезисы для защиты (10-15)

1. Главная идея проекта — перейти от точечной оценки NPV к распределению возможных исходов.
 2. Корреляция рисков важна, потому что плохие события в девелопменте часто наступают совместно, а не по отдельности.
 3. Модель с независимыми рисками систематически занижает риск и может быть слишком оптимистичной.
 4. Expected Shortfall показывает не просто границу потерь, а среднюю глубину плохих сценариев.
 5. Dashboard превращает теоретическую модель курсовой в прикладной инструмент инвестиционного анализа.
 6. Положительная средняя NPV не означает, что проект безопасен — важна вся форма распределения.
 7. Гауссова копула с разложением Холецкого — корректный способ задать совместное распределение разнородных рисков.
 8. Проверка матрицы на положительную определённость — это не формальность, а необходимое условие математической корректности модели.
 9. Разделение кода на UI, математику и утилиты делает проект тестируемым и расширяемым.
 10. Фиксированный seed обеспечивает воспроизводимость — любой может повторить мой результат.
 11. Tornado chart переводит абстрактную «чувствительность» в конкретный приоритет управления рисками.
 12. VaR говорит на одном языке с банками проектного финансирования.
 13. Модель честно обозначает свои ограничения и не выдаёт себя за инвестиционную рекомендацию.
 14. Векторизация на NumPy позволяет считать 10 000+ сценариев за доли секунды.
 15. Инструмент отвечает на главный вопрос инвестора: «с какой вероятностью я потеряю деньги и сколько?».
-

14. Возможные вопросы преподавателя и ответы (30)

1. **Почему выбран метод Монте-Карло?** Потому что он строит полное распределение NPV и позволяет оценить вероятность убытка, VaR и ES, чего не даёт ни точечный NPV, ни сценарный анализ.
2. **Почему нельзя ограничиться базовым NPV?** Точечная оценка не показывает риск: одинаковый NPV может скрывать совершенно разный разброс исходов.
3. **Почему риски считаются коррелированными?** В девелопменте рост затрат, задержки, падение цен и регуляторные ограничения часто наступают совместно; игнорировать это — значит занижать риск.

4. **Что такое корреляционная матрица?** Симметричная матрица 5×5 с единицами на диагонали, где элемент ρ_{ij} — сила и знак линейной связи между факторами.
5. **Что такое разложение Холецкого?** Представление матрицы $\Sigma = L \cdot L^T$; умножение независимых нормальных на L^T даёт величины с нужной корреляцией.
6. **Почему треугольное распределение?** Оно задаётся тремя понятными экспертными числами (min/мода/max) и не требует исторических данных — удобно для оценки проектов.
7. **Почему задержка задана дискретно?** Задержка — это исход в месяцах из небольшого набора (0/6/12), вероятности которого легко задать экспертно; непрерывное распределение здесь искусственно.
8. **Что такое VaR?** Уровень NPV, ниже которого результат окажется лишь с вероятностью 5% (5-й перцентиль распределения).
9. **Что такое Expected Shortfall?** Средняя NPV в худших 5% сценариев — средняя глубина потерь в зоне катастрофы.
10. **Чем VaR отличается от Expected Shortfall?** VaR — это граница («не хуже X в 95% случаев»), ES — среднее за этой границей; ES консервативнее и учитывает тяжесть хвоста.
11. **Что показывает tornado chart?** Ранжирование факторов риска по силе влияния на NPV (корреляция фактора с NPV).
12. **Что показывает heatmap?** Структуру попарных корреляций между факторами риска.
13. **Почему положительный NPV не означает безопасность?** Среднее не отражает разброс; при положительном среднем вероятность убытка может оставаться значительной.
14. **Как интерпретировать вероятность $NPV < 0$?** Это доля сценариев, в которых проект разрушает стоимость — прямая мера вероятности убытка.
15. **Что будет, если увеличить ставку дисконтирования?** Будущие потоки обесценятся сильнее, NPV снизится, распределение сместится влево, вероятность убытка вырастет.
16. **Что будет, если увеличить корреляцию рисков?** Левый хвост распределения станет тяжелее: вырастет $P(NPV < 0)$, ухудшатся VaR и ES при почти неизменном среднем.
17. **Почему модель учебная?** Параметры и корреляции заданы экспертно и условно, без калибровки на исторических данных.
18. **Какие ограничения у модели?** Условные параметры, отсутствие калибровки, упрощённый учёт долга/налогов; результат — не инвестиционная рекомендация.
19. **Как можно улучшить модель?** Калибровка распределений на рыночных данных, debt schedule, налоги, загрузка параметров из Excel, экспорт отчёта.
20. **Как использовать модель на практике?** Как инструмент предварительного риск-скрининга проекта и аргументации перед инвесткомитетом/банком.
21. **Кто целевая аудитория?** Девелоперы, инвесторы, банки проектного финансирования, оценщики, консультанты, студенты и преподаватели.
22. **Чем dashboard лучше обычной курсовой?** Это не текст, а работающий интерактивный инструмент: параметры можно менять и сразу видеть эффект.
23. **Чем dashboard лучше Excel-модели?** Корректная генерация коррелированных рисков через копулу, скорость (10 000+ сценариев), интерактивные графики, проверка матрицы на корректность, воспроизводимость.
24. **Можно ли применить модель к реальному проекту?** Да, после замены условных параметров и корреляций на калиброванные по реальным данным.

25. **Какие данные нужны для реального проекта?** Сметы и их историческая волатильность, рыночные цены и темпы продаж, статистика задержек, регуляторные издержки, обоснованные корреляции.
 26. **Почему Streamlit?** Позволяет на чистом Python быстро сделать интерактивный веб-интерфейс без фронтенд-разработки и бесплатно развернуть в облаке.
 27. **Почему Plotly?** Интерактивные графики (зум, подсказки), качественная визуализация распределений и тепловых карт.
 28. **Почему код разделён на app.py, model.py и utils.py?** Разделение ответственности: UI, математика и сервис независимы, что упрощает тестирование, чтение и развитие.
 29. **Что такое random seed?** Начальное состояние генератора случайных чисел; при одном seed выборка повторяется в точности.
 30. **Почему важна воспроизводимость?** Чтобы результат можно было проверить и повторить — это требование научной и инженерной добросовестности.
-

15. Ограничения модели

- параметры **условные** и заданы для демонстрации;
 - предельные распределения заданы **экспертно**, без статистической оценки;
 - корреляции заданы **экспертно**, не оценены на данных;
 - **нет калибровки** на исторических данных рынка недвижимости;
 - **проектное финансирование** (долг, график погашения) учтено упрощённо;
 - **налоги, debt schedule, escrow-счета** не детализированы;
 - результат **не является инвестиционной рекомендацией**.
-

16. Как улучшить проект

- загрузка параметров проекта из **Excel-файла**;
 - **экспорт результатов в PDF/отчёт**;
 - встроенные сценарии **base / optimistic / stress**;
 - модуль **debt schedule** (график обслуживания долга);
 - учёт **налогов** и escrow-счетов;
 - подключение **реальных рыночных данных** по недвижимости;
 - **калибровка распределений и корреляций** на исторических данных;
 - **авторизация пользователей и сохранение сценариев**;
 - постоянный **deployment на Streamlit Cloud** с публичной ссылкой.
-

17. Шпаргалка на 1 минуту

Я сделал интерактивный дашборд, который оценивает девелоперский проект методом Монте-Карло. Вместо одного значения NPV он строит распределение из десяти тысяч сценариев и показывает вероятность убытка, VaR и Expected Shortfall. Ключевая идея — учёт корреляции рисков: в девелопменте рост затрат, задержки и падение цен часто наступают совместно. Я сравниваю две модели — с независимыми и с коррелированными рисками — и показываю, что независимая модель занижает

риск, а корреляция утяжеляет левый хвост распределения. Коррелированные риски генерируются через гауссову копулу с разложением Холецкого, а матрица проверяется на положительную определённую. Код разделён на интерфейс, математику и утилиты. Главный вывод: проект помогает перейти от вопроса «чему равна NPV» к вопросу «с какой вероятностью проект создаст или разрушит стоимость».